

Sprint Documentation

Sprint 1	Start Date: 30/09/2025	Final Date: 06/10/2025
Project Name:	civitas-backend	
Ano: 4º	Análise e Desenvolvimento de Sistemas - AMS	
Team Members		
Felipe Pacheco Bianchini		
Wendell Alves Nascimento		
Giovanni da Silva Nascimento		

Sprint Backlog

Task #	Description	Start Date	Final date
012	Implementar os métodos padrão do CRUD para a entidade Secretaria, contemplando as seguintes operações: -Cadastrar -Alterar -Inativar e ativar registro -Listar	30/09/2025	06/10/2025

Task Description

Task #	Description	Assigned To	Status	Estimated Hours	Logged Hours
012	Criação do CRUD de Secretaria	Felipe Pacheco Bianchini, Wendell Alves Nascimento, Giovanni da Silva Nascimento	In Progress	10	10

Sprint Description

Implementar os métodos padrão do CRUD para a entidade secretaria, contemplando as seguintes operações:

- Cadastrar
- Alterar

- Inativar e ativar registro
- Listar

Sprint Results

Durante a Sprint 01, a equipe se dedicou integralmente à tarefa de "Criação do CRUD de Secretaria", que constituiu o escopo completo deste ciclo de desenvolvimento. O objetivo foi estabelecer a primeira funcionalidade essencial do backend, permitindo o gerenciamento completo dos dados de secretarias. O trabalho foi executado em camadas, seguindo boas práticas de arquitetura de software.

Secretaria
- idSecretaria : int
- situacao : int
- descricao : String
- cnpj : String
- nome : String
- logradouro : String
- numero : String
- bairro : String
- cep : String
- nomeRazaoSocial : String
- telefone : String
- email : String
- cidade : String
- estado : String

As entregas da tarefa foram:

Estrutura da Entidade e Banco de Dados: Foi criada a classe de modelo `Secretaria.cs` para representar a entidade no sistema. Com base nesse modelo, foi gerada e aplicada uma migration (`InitialCreateSecretaria`) utilizando o Entity Framework Core, que criou a tabela "Secretarias" no banco de dados PostgreSQL. A estrutura da tabela inclui validações importantes, como a unicidade dos campos `Cnpj` e `Email`, garantindo a integridade dos dados.

Camada de Acesso a Dados (Repository): Foi implementado o `SecretariaRepository.cs`, que abstrai e centraliza toda a comunicação com o banco de dados para a entidade `Secretaria`. Esta camada é responsável por executar as operações de consulta, inserção, atualização e exclusão, seguindo o Repository Pattern.

Camada de Regras de Negócio (Service): A classe `SecretariaService.cs` foi desenvolvida para orquestrar as operações e conter as regras de negócio. Uma das principais responsabilidades implementadas nesta camada é a validação para impedir o cadastro de secretarias com CNPJ ou email duplicados, consultando o repositório antes de efetivar uma criação ou alteração.

Mapeamento de Objetos e DTOs: Foram criados os Data Transfer Objects (DTOs) na pasta `DTOs/SecretariaDto.cs` para desacoplar a representação dos dados da API do modelo de dados do banco. A biblioteca AutoMapper foi configurada no arquivo `Mappings/SecretariaProfile.cs` para automatizar a conversão entre os modelos e os DTOs.

API Endpoints (Controller): A classe SecretariaController.cs foi criada para expor as funcionalidades via uma API RESTful. Foram implementados endpoints para as seguintes operações:

Listar todas as secretarias (GET /api/secretaria).

Listar apenas secretarias ativas (GET /api/secretaria/active).

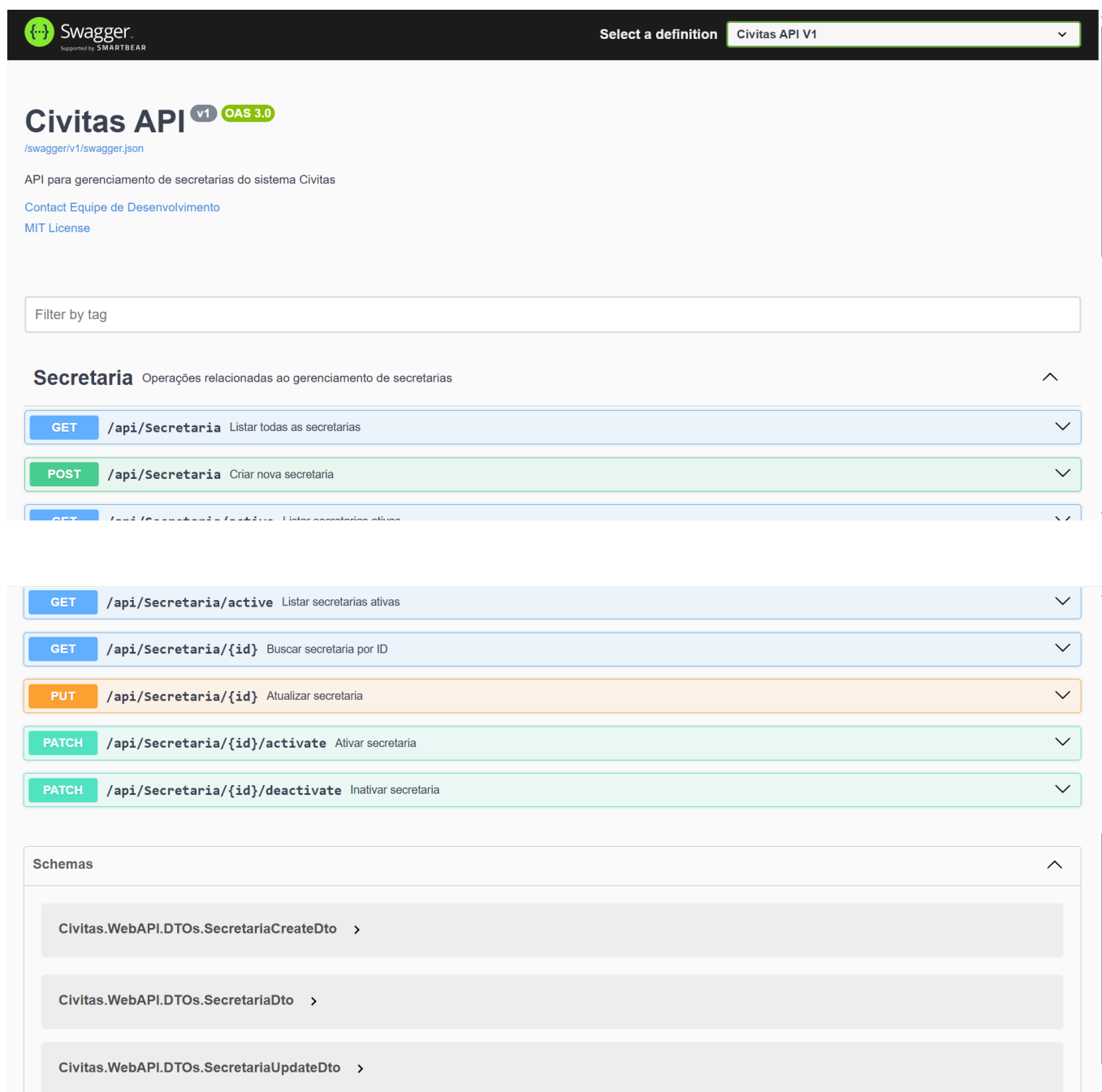
Obter uma secretaria por seu ID (GET /api/secretaria/{id}).

Cadastrar uma nova secretaria (POST /api/secretaria).

Alterar uma secretaria existente (PUT /api/secretaria/{id}).

Ativar uma secretaria (PATCH /api/secretaria/{id}/activate).

Inativar uma secretaria (PATCH /api/secretaria/{id}/deactivate).



The image shows the Swagger UI for the Civitas API V1. The interface is dark-themed with a black header. The Swagger logo is on the left, and a dropdown menu on the right shows 'Civitas API V1' selected. Below the header, the API title 'Civitas API' is displayed with a 'v1' tag and an 'OAS 3.0' badge. A link to the swagger.json file is provided. The description states it's an API for managing secretaries. A 'Filter by tag' input field is present. The 'Secretaria' section is expanded, showing a list of endpoints with their methods, paths, and descriptions. The endpoints are: GET /api/Secretaria (Listar todas as secretarias), POST /api/Secretaria (Criar nova secretaria), GET /api/Secretaria/{id} (Buscar secretaria por ID), GET /api/Secretaria/active (Listar secretarias ativas), PUT /api/Secretaria/{id} (Atualizar secretaria), PATCH /api/Secretaria/{id}/activate (Ativar secretaria), and PATCH /api/Secretaria/{id}/deactivate (Inativar secretaria). Below the endpoints, the 'Schemas' section is expanded, showing three schemas: Civitas.WebAPI.DTOs.SecretariaCreateDto, Civitas.WebAPI.DTOs.SecretariaDto, and Civitas.WebAPI.DTOs.SecretariaUpdateDto.

Swagger
Supported by SMARTBEAR

Select a definition Civitas API V1

Civitas API v1 OAS 3.0

/swagger/v1/swagger.json

API para gerenciamento de secretarias do sistema Civitas

[Contact Equipe de Desenvolvimento](#)

[MIT License](#)

Filter by tag

Secretaria

Operações relacionadas ao gerenciamento de secretarias

- GET** /api/Secretaria Listar todas as secretarias
- POST** /api/Secretaria Criar nova secretaria
- GET** /api/Secretaria/{id} Buscar secretaria por ID
- GET** /api/Secretaria/active Listar secretarias ativas
- PUT** /api/Secretaria/{id} Atualizar secretaria
- PATCH** /api/Secretaria/{id}/activate Ativar secretaria
- PATCH** /api/Secretaria/{id}/deactivate Inativar secretaria

Schemas

- Civitas.WebAPI.DTOs.SecretariaCreateDto
- Civitas.WebAPI.DTOs.SecretariaDto
- Civitas.WebAPI.DTOs.SecretariaUpdateDto

A execução da tarefa foi bem-sucedida, resultando em uma funcionalidade robusta e bem-estruturada. Os principais aprendizados e pontos positivos foram:

Arquitetura em Camadas: A implementação seguiu uma arquitetura limpa com separação de responsabilidades (Controller, Service, Repository). Isso não apenas organiza o código, mas também facilita a manutenção futura e a implementação de testes unitários.

Validações de Negócio: A implementação de validações cruciais, como a checagem de unicidade de CNPJ e email no nível de serviço, garante a consistência dos dados e a robustez da aplicação contra dados inválidos.

Uso de Ferramentas do Ecossistema .NET: A equipe demonstrou proficiência no uso de ferramentas como Entity Framework Core para migrações de banco de dados e AutoMapper para mapeamento de objetos, otimizando o tempo de desenvolvimento.

A tarefa "Sprint 01 – Task 012 - Criação do CRUD de Secretaria" foi totalmente concluída conforme o escopo definido. Não houve impedimentos ou funcionalidades que deixaram de ser implementadas.

Para garantir a qualidade e o correto funcionamento da API, foram realizados testes de integração para cada um dos endpoints desenvolvidos. O arquivo Civitas.WebAPI.http contém a suíte de testes utilizada, que validou os seguintes cenários:

Cenários de Sucesso: Foram testadas todas as operações de CRUD (criação, leitura, atualização e ativação/inativação) com dados válidos, verificando se os códigos de status HTTP (200, 201) e os dados retornados estavam corretos.

Cenários de Erro: Foram realizados testes específicos para as regras de negócio, como a tentativa de cadastrar uma secretaria com um CNPJ ou email já existente. Conforme esperado, a API retornou o código de status HTTP 409 (Conflict), validando a lógica de tratamento de duplicidade.

Cenário de Não Encontrado: Foi testada a busca por uma secretaria com um ID inexistente, e a API retornou corretamente o código de status HTTP 404 (Not Found).

Assinaturas